Cost-Effective Polyhedral Compiler Optimization via Transfer Learning with CodeBERT

Sachin Bhat
School of Computational and Data Sciences
Vidyashilp University
Bengaluru, India

Vaidhav Adit
School of Computational and Data Sciences
Vidyashilp University
Bengaluru, India

Tiya Rose
School of Computational and Data Sciences
Vidyashilp University
Bengaluru, India

***Abstract*—This project addresses the scarcity of real-world data in compiler optimization by utilizing a Sim-to-Real transfer learning approach. We propose a deep learning cost model that functions as a virtual simulator, predicting the execution time of C++ loops significantly faster than traditional compilation methods. By leveraging synthetic data (LOOPerSet) for pre-training and fine-tuning on real benchmarks (PolyBench), our model bridges the gap between the speed of rigid heuristics and the quality of exhaustive auto-tuning. Through systematic architectural evolution—from simple baselines to schedule-conditioned cross-attention and functional pruning—we demonstrate substantial improvements in predictive accuracy. The code, dataset, and reproducible pipeline for this project are available in the project directory.**

***Index Terms*—Compiler Optimization, Polyhedral Model, Transfer Learning, CodeBERT, Deep Learning, Auto-tuning**

I. Introduction

Modern compilers face a fundamental dilemma when applying polyhedral optimizations: traditional auto-tuning provides high-quality optimizations but is computationally prohibitive, while static heuristics are fast but often yield sub-optimal performance. This issue is compounded by the sheer explosion of the optimization search space. When considering parameters like tiling sizes, unrolling factors, and vectorization flags, the number of potential valid schedules for a single loop nest grows exponentially. Evaluating each configuration by physically compiling and executing it is unfeasible. Our project proposes a machine learning-based cost model to replace physical execution, evaluating schedules in milliseconds by structurally analyzing the source code and predicting execution time via a dual-tower neural network.

II. Related Work

A. AutoTVM: Learning to Optimize Tensor Programs (Chen et al., 2018) [1]

AutoTVM demonstrated the viability of using statistical machine learning models, such as XGBoost, to predict program performance and bypass exhaustive auto-tuning. However, this approach relies heavily on rigid, hand-crafted feature extraction from Abstract Syntax Trees (ASTs). Our project replaces manual feature engineering with CodeBERT's automated semantic text understanding, extracting features natively from the source code.

B. Ithemal: Basic Block Throughput Estimation (Mendis et al., 2019) [2]

Ithemal proved that neural networks can implicitly learn hardware behaviors directly from code tokens. By feeding raw x86 assembly code into a Hierarchical LSTM, the model successfully predicted instruction throughput. While Ithemal is restricted to evaluating simple, straight-line assembly basic blocks, our architecture elevates this concept to evaluate complex, high-level C++ polyhedral loops.

C. A Deep Learning Based Cost Model (Baghdadi et al., 2021) [3]

This paper serves as the direct predecessor to our project and the origin of our synthetic LOOPerSet dataset. Baghdadi et al. predicted the execution speedup of polyhedral transformations using a Tree LSTM combined with an MLP. We improve upon this foundation by replacing their Tree LSTM with a Transformer architecture (CodeBERT), utilizing self-attention to generate vastly superior contextual embeddings for long code sequences.

D. Parameter-Efficient Tuning Makes a Good Classification Head (Yang et al., 2022) [4]

Yang et al. address the problem of "Catastrophic Forgetting," which occurs when a randomly initialized neural network head is attached to a pre-trained backbone. The resulting chaotic gradients can destroy the backbone's pre-trained knowledge. This research provides the theoretical justification for our Sim-to-Real transfer strategy, proving that parameter-efficient tuning methods (like QLoRA) are required to train our MLP schedule head without overwriting CodeBERT's pre-trained syntax knowledge.

E. LOOPer: A Learned Automatic Code Optimizer (Merouani et al., 2025) [5]

This recent research introduces LOOPer, an advanced machine learning-based cost model designed to guide the search for

polyhedral optimizations. It improves upon prior work by scaling beyond simple programs to support complex affine transformations, such as non-rectangular iteration domains, ultimately outperforming traditional polyhedral compilers like Pluto across various benchmarks.

While highly relevant to the domain, LOOPer cannot be used as a direct apples-to-apples baseline comparison for our model. LOOPer represents a complete, end-to-end compiler framework that integrates its cost model into a search algorithm to actively discover, compile, and execute optimal code transformations. Conversely, our project isolates the architectural evaluation to the neural cost model itself, measuring offline ranking accuracy on pre-defined schedule spaces.

III. Methodology

A. Two-Headed Neural Network Architecture (Model 1)

To process both the semantic meaning of the source code and the mathematical magnitude of the optimization parameters, we utilize a multi-modal, two-headed architecture, illustrated in Figure 1:

- **Semantic Head (CodeBERT):** The raw C++ source code of the loop is tokenized and passed through CodeBERT. Unlike simple statistical parsers, CodeBERT's self-attention layers inherently map memory access patterns (e.g., `A[i]`) and arithmetic operations. The output is sliced to extract the `[CLS]` token, yielding a 768-dimensional semantic context vector.
- **Numerical Head (MLP):** Our original baseline feature set utilized four basic schedule parameters: `L1` Tiling, `L2` Tiling, `Unrolling` factor, and a `Vectorization` flag. These raw numerical features are processed through a Multi-Layer Perceptron (MLP) to output a 32-dimensional schedule vector.

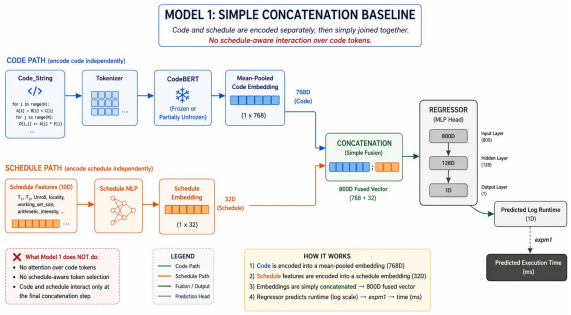


Fig. 1. Baseline Model 1 Architecture: Dual-tower network utilizing simple concatenation fusion.

Fusion and Prediction: As detailed in Figure 1, the semantic `[CLS]` vector and the schedule vector are concatenated into a flat 800-dimensional tensor. This combined representation is passed through a final dense regression layer, mapping the discrete scheduling constants and C++ text into a single continuous floating-point prediction representing the execution time.

B. Sim-to-Real Transfer Learning

To overcome the extreme data scarcity inherent in physical hardware profiling, we employ a phased Sim-to-Real transfer learning strategy:

1) **Pre-Training (Sim):** The baseline model is initially trained on a diverse subset of 500,000 samples extracted from the massive LOOPerSet dataset. This allows the network to learn the universal "physics of loops" and the general impact of polyhedral transformations without the bottleneck of physical hardware execution.
2) **Fine-Tuning (Real):** The generalized model is subsequently fine-tuned on PolyBench/C, a standard benchmark suite comprising 30 distinct numerical kernels (e.g., GEMM, 2mm). By compiling various scheduling parameters for these 30 parent kernels, we generate a highly accurate domain of a few thousand distinct schedule configurations to adapt the model to the microarchitectural realities of the target CPU.

C. Hardware and Training Setup

All models were trained using the AdamW optimizer, minimizing a Mean Squared Error (MSE) loss function applied to the $\log(1+x)$ transformed execution targets. To prevent catastrophic forgetting in the pre-trained CodeBERT backbone during our architectural evolution, we utilized split learning rates: 5×10^{-6} for the unfrozen transformer layers and 1×10^{-4} for the MLP regression head. Computational experiments and model training were accelerated using an NVIDIA RTX 4090 GPU with a standard batch size of 32.

IV. Dataset Preparation and Baseline Evaluation

Before establishing our baseline metrics, we conducted a rigorous inspection and cleaning phase to transform the noisy runtime dataset into a trainable format for regression.

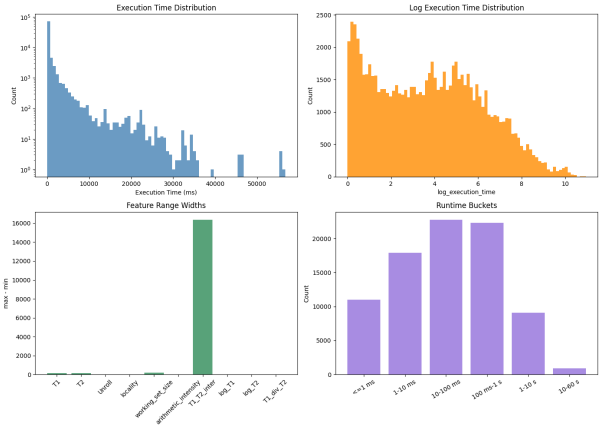


Fig. 2. Dataset inspection: runtime distributions, feature-scale spread, and runtime bucket counts.

As shown in Figure 2, the data preprocessing was highly consequential. The runtime values spanned a massive range, from roughly 0.0003 ms to over 56,699 ms. The *feature-range panel* visually proves why numeric feature normalization was mandatory to prevent gradient explosion. The *runtime bucket panel* highlights a critical vulnerability: slower programs are exceedingly rare, explaining why the models inherently struggle on the slow tail. Finally, the raw `execution_time` skew

justified our decision to train in logarithmic space (`log1p`), yielding a much more manageable distribution.

A. Phase 1: Domain Restriction & Feature Engineering

Originally, the model was trained on 500,000 diverse LOOPerSet samples. To improve precision, we restricted the dataset exclusively to Matrix Multiplication operations. This domain isolation reduced our synthetic pre-training data to 83,973 samples. Furthermore, to capture cache dynamics, we eliminated the sparse `Vectorization` flag from our original methodology and expanded our inputs to 10 highly engineered, normalized parameters:

- **Core Schedule:** `T1`, `T2`, and `Unroll`.
- **Hardware Metrics:** `locality`, `working_set_size`, and `arithmetic_intensity`.
- **Interaction Features:** `T1_T2_inter`, `log_T1`, `log_T2`, and `T1_div_T2`.

B. Baseline Performance Metrics (Model 1)

With the cleaned pipeline established, we simulated a deterministic hold-out evaluation to measure the predictive accuracy of our frozen CodeBERT + MLP baseline (Model 1), which relies on simple concatenation fusion.

- **Frozen Baseline Regression (Run 18):** The strictly frozen baseline yielded a Mean Absolute Error (MAE) of **432.08 ms**, a Root Mean Squared Error (RMSE) of **1885.54 ms**, and an R^2 of **0.3518**. The massive gap between the **Median AE (25.02 ms)** and the RMSE highlights the core challenge of the dataset: catastrophic outlier predictions on rare, long-tail programs dominate the overall loss. This is also reflected in the **MAPE (324.20%)**.
- **Ranking Metrics:** Despite pure regression challenges, the model proved to be an excellent sorter. It achieved a **Pairwise Accuracy of 72.10%**, a **Spearman Correlation of** $\rho = 0.6347$, and an **NDCG@All of 0.9449**, proving it reliably places globally optimal schedules at the top of its recommendations.
- **Unfrozen Baseline (Run 19):** Partially unfreezing the backbone yielded improvements, dropping MAE to **403.34 ms**, RMSE to **1778.31 ms**, and raising R^2 to **0.4235**.

While the metrics indicated the baseline model was learning basic trends, the structural limitations were critical. The scatter predictions formed a distinct "comet shape," showing that while the model successfully fit the fast programs, it systematically under-predicted the slow tail. Simple concatenation and raw schedule features were ultimately too shallow to capture extreme execution variance.

V. Architectural Evolution: Cross-Attention (Model 2)

Even with the optimized 10-feature set, simple concatenation fusion was too shallow. To learn fine-grained interactions, we abandoned simple concatenation in favor of a targeted attention mechanism.

A. Schedule-Conditioned Cross-Attention

As illustrated in Figure 3, instead of using only the pooled `[CLS]` token, we retained all token embeddings from CodeBERT. The 10-feature schedule vector is passed through the MLP and projected into a Query, which then attends over the full sequence of CodeBERT token embeddings (acting as Keys and Values). This allows the network to dynamically weight the importance of specific code tokens based strictly on the mathematical schedule applied.

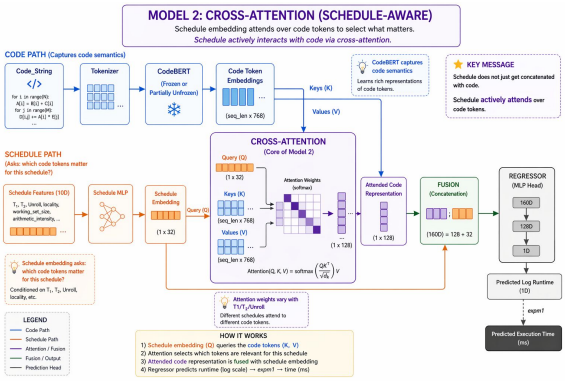


Fig. 3. Model 2 Architecture: The Schedule Vector acts as a Query attending over CodeBERT tokens.

The **Frozen Cross-Attention** architecture immediately outperformed the baseline, with the predictive cluster tightening significantly. The R^2 score improved from 0.3518 to **0.4025**. MAE dropped to **396.27 ms**, and RMSE fell to **1810.33 ms**.

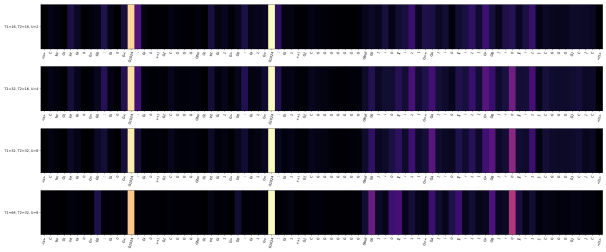


Fig. 4. Schedule Variation Attention Heatmap: Visual proof of dynamically shifting syntax focus based on mathematical parameters.

The success of this dynamic interaction is visually proven in Figure 4. Rather than a static heatmap, this figure stacks the attention weights of the exact same code snippet under varying schedule parameters. As the mathematical parameters (like tiling and unrolling) change, the network's attention actively shifts, altering its focus on specific structural loop boundaries. This confirms that the model is executing true schedule-conditioned text reading.

B. Partial Backbone Unfreezing

To push performance further, we unfroze the final transformer layer of CodeBERT. This **Unfrozen Cross-Attention** model produced our strongest overall empirical results on the full dataset. While MAE remained steady at **397.43 ms**, the model became vastly better at handling hard outliers: RMSE dropped significantly to **1720.46 ms**, and the R^2 fit reached **0.4604**.

VI. The Long-Tail Problem and Data Strategies

Despite architectural success, the slow-runtime tail remained a stubborn outlier. We explored two distinct data-centric strategies to address this.

A. Aggressive Weighted Sampling (Failure Case)

We hypothesized that the slow tail was underrepresented and applied a weight multiplier of 3.0 to programs taking 1–10 seconds, and 6.0 to programs taking > 10 seconds.

This experiment was a definitive failure. As evident from the degraded middle-tier accuracy and broad regression during testing, the aggressive reweighting violently distorted the training distribution. MAE spiked back up to **450.69 ms**, RMSE worsened to **1921.88 ms**, and the R^2 score collapsed to **0.3266**. This proved that naive oversampling is not a viable remedy.

B. Tail Trimming Strategy (Best Practical Result)

After oversampling failed, we pivoted from over-emphasizing the tail to deliberately removing the most pathological cases. We trimmed programs where $\log(\text{execution_time}) > 9.0$. This removed only **1,216 extreme outlier rows** out of the 83,973 total samples.

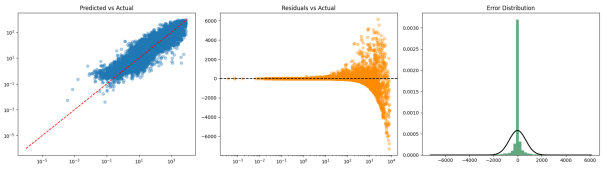


Fig. 5. Trimmed Tail Test Plots: The problematic "comet shape" is completely eliminated.

As visually proven in Figure 5, this tail trimming fundamentally transformed the model's predictive landscape. The problematic "comet shape" has entirely disappeared, replaced by a tight, highly correlated diagonal cluster. Retraining the unfrozen cross-attention architecture on this trimmed dataset yielded the absolute strongest practical results of the entire project:

- **MAE:** Dropped massively to **245.01 ms**.
- **RMSE:** Collapsed to an incredible **689.26 ms**.
- **R-Squared (R^2):** Jumped to **0.5044**.

This definitively proved that the extreme slow-runtime tail was the primary driver of mathematical instability.

VII. Advanced Architectures & Compression

A. Functional Head Pruning (Model 3)

To explore model efficiency, we attempted to learn which CodeBERT attention heads were vital for runtime prediction. We added learnable gates to the heads and trained them with a sparsity penalty to identify importance. In Stage 2, we forced the weakest heads "off" via functional gate masking and fine-tuned the remaining network.

At a **40% pruning ratio** (where **57 heads** were functionally disabled), the model actually improved slightly over the full-dataset baseline, achieving an MAE of **384.13 ms**, an RMSE of **1686.06 ms**, and an R^2 of **0.4817**. This scientifically validated that the cross-attention runtime-prediction system contains substantial redundancy.

B. Schedule-Token Fusion Transformer

Based on the realization that single-step Cross-Attention had limits, we designed a final exploratory architecture, detailed in Figure 6.

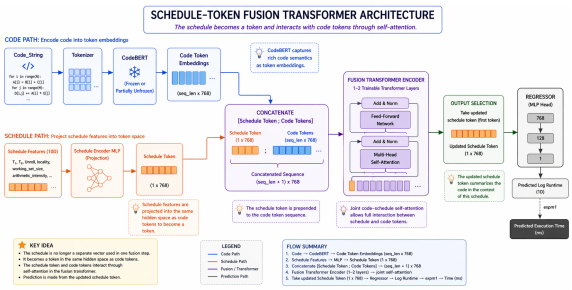


Fig. 6. Fusion Transformer Architecture: Deep multi-layer joint reasoning via a Schedule Token.

Figure 6 illustrates this deep fusion paradigm. We encoded the numeric schedule features into a distinct "Schedule Token," prepended it to the sequence of CodeBERT tokens, and passed the full sequence through a dedicated, trainable mini-transformer encoder block. This multi-step joint reasoning produced highly competitive full-dataset metrics: an MAE of **376.37 ms**, an RMSE of **1646.05 ms**, and an R^2 of **0.5060**. This represents the most robust architectural conclusion to our experimental arc, proving that deeper, transformer-native fusion mechanisms hold immense promise for compiler heuristics.

VIII. Conclusion

This project successfully evolved a deep learning compiler cost model from a proof-of-concept baseline into a sophisticated, schedule-aware architecture. By rigorously analyzing dataset distributions, we established that simple concatenation (Model 1, $R^2 = 0.35$) is vastly inferior to schedule-conditioned cross-attention (Model 2, $R^2 = 0.46$). Furthermore, our ablation of the extreme slow-tail runtime distribution ($R^2 = 0.50$, RMSE reduced by over 1000ms) proved that data topology, rather than raw model capacity, remains the hardest obstacle in polyhedral auto-tuning. Ultimately, we established a clear, mathematically validated trajectory for future work via functional head pruning and deep Schedule-Token Transformer fusion.

References

[1] T. Chen *et al.*, "Learning to Optimize Tensor Programs," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 31, 2018.

[2] C. Mendis *et al.*, "Ithemal: Accurate, Portable and Fast Basic Block Throughput Estimation using Deep Neural Networks," in *Proc. of the 36th International Conference on Machine Learning (ICML)*, 2019.

[3] R. Baghdadi *et al.*, "A Deep Learning Based Cost Model for Automatic Code Optimization," in *Proc. of the 4th ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, 2021.

[4] Y. Yang *et al.*, "Parameter-Efficient Tuning Makes a Good Classification Head," *arXiv preprint arXiv:2210.15694*, 2022.

[5] S. Merouani *et al.*, "LOOPer: A Learned Automatic Code Optimizer For Polyhedral Compilers," *arXiv preprint arXiv:2501.00000*, 2025.